

BAI Work 技术白皮书

BAI Work Contributors

2026 年 7 月

版本说明

本白皮书是 BAI Work 项目的第一版完整技术白皮书。内容基于当前仓库实现、项目 README、构建配置、运行时适配代码、资源目录说明、EBAI 映射服务以及公开 BAI Code 安装资料整理。文档采用保守口径：能从代码和资源核对的内容写为当前事实，官方没有公开的 BAI Code 桌面本地 HTTP service/session/event/permission/question 协议写为待确认边界。

本文不包含真实 API key、token 或任何凭据。涉及环境变量的示例均使用占位符，例如 BAI_API_KEY=your-key。

目录

第一章	产品定位与白皮书边界	1
1.1	本章摘要	1
1.2	背景与目标	1
1.3	仓库证据	2
1.4	关键设计	2
1.5	目标用户与使用场景	3
1.6	实现说明	4
1.7	安全与合规考量	4
1.8	运维与发布影响	5
1.9	边界、限制与后续演进	5
1.10	工程判定清单	6
1.11	专项展开: 品牌治理	6
1.12	维护者补充说明	7
1.13	验收与交付说明	8
1.14	术语一致性说明	8
1.15	更新原则	9
1.16	小结	9
第二章	系统总览与分层架构	10
2.1	本章摘要	10
2.2	背景与目标	10
2.3	仓库证据	11
2.4	关键设计	11
2.5	目标用户与使用场景	12
2.6	实现说明	13
2.7	安全与合规考量	13
2.8	运维与发布影响	14
2.9	边界、限制与后续演进	14
2.10	工程判定清单	15

目录	iii
2.11 专项展开: 分层协作	15
2.12 维护者补充说明	16
2.13 验收与交付说明	17
2.14 术语一致性说明	17
2.15 更新原则	18
2.16 小结	18
第三章 Electron/Vite/React 主壳	19
3.1 本章摘要	19
3.2 背景与目标	19
3.3 仓库证据	20
3.4 关键设计	20
3.5 目标用户与使用场景	21
3.6 实现说明	22
3.7 安全与合规考量	22
3.8 运维与发布影响	23
3.9 边界、限制与后续演进	23
3.10 工程判定清单	24
3.11 专项展开: 界面与边界	24
3.12 维护者补充说明	25
3.13 验收与交付说明	26
3.14 术语一致性说明	26
3.15 更新原则	27
3.16 小结	27
第四章 BAI Code Runtime 适配策略	28
4.1 本章摘要	28
4.2 背景与目标	28
4.3 仓库证据	29
4.4 关键设计	29
4.5 目标用户与使用场景	30
4.6 实现说明	31
4.7 安全与合规考量	31
4.8 运维与发布影响	32
4.9 边界、限制与后续演进	32
4.10 工程判定清单	33
4.11 专项展开: 桥接策略	33
4.12 维护者补充说明	34

4.13	验收与交付说明	35
4.14	术语一致性说明	35
4.15	更新原则	36
4.16	小结	36
第五章	官方 Wheelhouse 与本地 venv	37
5.1	本章摘要	37
5.2	背景与目标	37
5.3	仓库证据	38
5.4	关键设计	38
5.5	目标用户与使用场景	39
5.6	实现说明	40
5.7	安全与合规考量	40
5.8	运维与发布影响	41
5.9	边界、限制与后续演进	41
5.10	工程判定清单	42
5.11	专项展开: 依赖策略	42
5.12	维护者补充说明	43
5.13	验收与交付说明	44
5.14	术语一致性说明	44
5.15	更新原则	45
5.16	小结	45
第六章	Mac Intel x64 Runtime 打包	46
6.1	本章摘要	46
6.2	背景与目标	46
6.3	仓库证据	47
6.4	关键设计	47
6.5	目标用户与使用场景	48
6.6	实现说明	49
6.7	安全与合规考量	49
6.8	运维与发布影响	50
6.9	边界、限制与后续演进	50
6.10	工程判定清单	51
6.11	专项展开: 嵌入式运行时	51
6.12	维护者补充说明	52
6.13	验收与交付说明	52
6.14	术语一致性说明	53

6.15 更新原则	54
6.16 小结	54
第七章 跨平台构建与发布	55
7.1 本章摘要	55
7.2 背景与目标	55
7.3 仓库证据	56
7.4 关键设计	56
7.5 目标用户与使用场景	57
7.6 实现说明	58
7.7 安全与合规考量	58
7.8 运维与发布影响	59
7.9 边界、限制与后续演进	59
7.10 工程判定清单	60
7.11 专项展开: 产物治理	60
7.12 维护者补充说明	61
7.13 验收与交付说明	62
7.14 术语一致性说明	62
7.15 更新原则	63
7.16 小结	63
第八章 配置模型与设置迁移	64
8.1 本章摘要	64
8.2 背景与目标	64
8.3 仓库证据	65
8.4 关键设计	65
8.5 目标用户与使用场景	66
8.6 实现说明	67
8.7 安全与合规考量	67
8.8 运维与发布影响	68
8.9 边界、限制与后续演进	68
8.10 工程判定清单	69
8.11 专项展开: 兼容迁移	69
8.12 维护者补充说明	70
8.13 验收与交付说明	71
8.14 术语一致性说明	71
8.15 更新原则	72
8.16 小结	72

第九章 Provider 设置与模型能力	73
9.1 本章摘要	73
9.2 背景与目标	73
9.3 仓库证据	74
9.4 关键设计	74
9.5 目标用户与使用场景	75
9.6 实现说明	76
9.7 安全与合规考量	76
9.8 运维与发布影响	77
9.9 边界、限制与后续演进	77
9.10 工程判定清单	78
9.11 专项展开: 模型治理	78
9.12 维护者补充说明	79
9.13 验收与交付说明	80
9.14 术语一致性说明	80
9.15 更新原则	81
9.16 小结	81
第十章 本地服务边界与协议现状	82
10.1 本章摘要	82
10.2 背景与目标	82
10.3 仓库证据	83
10.4 关键设计	83
10.5 目标用户与使用场景	84
10.6 实现说明	85
10.7 安全与合规考量	85
10.8 运维与发布影响	86
10.9 边界、限制与后续演进	86
10.10 工程判定清单	87
10.11 专项展开: 协议边界	87
10.12 维护者补充说明	88
10.13 验收与交付说明	89
10.14 术语一致性说明	89
10.15 更新原则	90
10.16 小结	90
第十一章 安全边界与凭据保护	91
11.1 本章摘要	91

11.2 背景与目标	91
11.3 仓库证据	92
11.4 关键设计	92
11.5 目标用户与使用场景	93
11.6 实现说明	94
11.7 安全与合规考量	94
11.8 运维与发布影响	95
11.9 边界、限制与后续演进	95
11.10 工程判定清单	96
11.11 专项展开: 凭据与权限	96
11.12 维护者补充说明	97
11.13 验收与交付说明	97
11.14 术语一致性说明	98
11.15 更新原则	99
11.16 小结	99
第十二章 EBAI 映射	100
12.1 本章摘要	100
12.2 背景与目标	100
12.3 仓库证据	101
12.4 关键设计	101
12.5 目标用户与使用场景	102
12.6 实现说明	103
12.7 安全与合规考量	103
12.8 运维与发布影响	104
12.9 边界、限制与后续演进	104
12.10 工程判定清单	105
12.11 专项展开: 生态迁移	105
12.12 维护者补充说明	106
12.13 验收与交付说明	107
12.14 术语一致性说明	107
12.15 更新原则	108
12.16 小结	108
第十三章 Hooks 安全策略	109
13.1 本章摘要	109
13.2 背景与目标	109
13.3 仓库证据	110

13.4 关键设计	110
13.5 目标用户与使用场景	111
13.6 实现说明	112
13.7 安全与合规考量	112
13.8 运维与发布影响	113
13.9 边界、限制与后续演进	113
13.10 工程判定清单	114
13.11 专项展开: 执行治理	114
13.12 维护者补充说明	115
13.13 验收与交付说明	116
13.14 术语一致性说明	116
13.15 更新原则	117
13.16 小结	117
第十四章 工程 workflow 表面	118
14.1 本章摘要	118
14.2 背景与目标	118
14.3 仓库证据	119
14.4 关键设计	119
14.5 目标用户与使用场景	120
14.6 实现说明	121
14.7 安全与合规考量	121
14.8 运维与发布影响	122
14.9 边界、限制与后续演进	122
14.10 工程判定清单	123
14.11 专项展开: 用户路径	123
14.12 维护者补充说明	124
14.13 验收与交付说明	124
14.14 术语一致性说明	125
14.15 更新原则	126
14.16 小结	126
第十五章 写作、自动化与多模态扩展	127
15.1 本章摘要	127
15.2 背景与目标	127
15.3 仓库证据	128
15.4 关键设计	128
15.5 目标用户与使用场景	129

15.6 实现说明	130
15.7 安全与合规考量	130
15.8 运维与发布影响	131
15.9 边界、限制与后续演进	131
15.10 工程判定清单	132
15.11 专项展开: 写作与自动化	132
15.12 维护者补充说明	133
15.13 验收与交付说明	133
15.14 术语一致性说明	134
15.15 更新原则	135
15.16 小结	135
第十六章 诊断、测试与可维护性	136
16.1 本章摘要	136
16.2 背景与目标	136
16.3 仓库证据	137
16.4 关键设计	137
16.5 目标用户与使用场景	138
16.6 实现说明	139
16.7 安全与合规考量	139
16.8 运维与发布影响	140
16.9 边界、限制与后续演进	140
16.10 工程判定清单	141
16.11 专项展开: 可维护性	141
16.12 维护者补充说明	142
16.13 验收与交付说明	143
16.14 术语一致性说明	143
16.15 更新原则	144
16.16 小结	144
第十七章 已知限制与风险登记	145
17.1 本章摘要	145
17.2 背景与目标	145
17.3 仓库证据	146
17.4 关键设计	146
17.5 目标用户与使用场景	147
17.6 实现说明	148
17.7 安全与合规考量	148

17.8 运维与发布影响	149
17.9 边界、限制与后续演进	149
17.10 工程判定清单	150
17.11 专项展开: 风险登记	150
17.12 维护者补充说明	151
17.13 验收与交付说明	151
17.14 术语一致性说明	152
17.15 更新原则	153
17.16 小结	153
第十八章 路线图与演进原则	154
18.1 本章摘要	154
18.2 背景与目标	154
18.3 仓库证据	155
18.4 关键设计	155
18.5 目标用户与使用场景	156
18.6 实现说明	157
18.7 安全与合规考量	157
18.8 运维与发布影响	158
18.9 边界、限制与后续演进	158
18.10 工程判定清单	159
18.11 专项展开: 演进原则	159
18.12 维护者补充说明	160
18.13 验收与交付说明	161
18.14 术语一致性说明	161
18.15 更新原则	162
18.16 小结	162
参考资料	163

第一章 产品定位与白皮书边界

1.1 本章摘要

本章说明产品定位在 BAI Work 技术体系中的位置。核心关键词包括桌面智能体工作台、BAI/BAI Code 品牌统一、从副本独立后的边界治理、不杜撰未公开协议。本章内容以仓库当前实现为准,以公开 BAI Code 安装资料为边界,不把未公开的桌面本地 HTTP 协议写成既定官方契约。

1.2 背景与目标

本章聚焦产品定位。BAI Work 的设计起点不是重新发明一个孤立桌面客户端,而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理,对外使用 BAI Work 名称,对运行时使用 BAI Code 叙事,对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换:用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用,维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

产品定位的目标可以概括为三点。第一,保持桌面体验稳定,让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二,尊重 BAI Code 公开文档的边界,公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数,但没有公开桌面本地 HTTP service/session/event/permission/question 协议,因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三,将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束,便于后续团队按相同标准扩展。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI,用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code,面向维护者时说明这些兼容接口的边界,防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill,复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要,因为规则包来自外部仓库,如果允许符号链接,可能把用户不期待的本地路径打包到技能目录。

1.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

1.4 关键设计

在产品定位中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括：桌面智能体工作台、BAI/BAI Code 品牌统一、从副本独立后的边界治理、不杜撰未公开协议。这些重点之间存在明确依赖关系：打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

1.5 目标用户与使用场景

软件工程师、技术负责人、企业内部工具团队、打包发布维护者是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同：文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

1.6 实现说明

当前实现中, 产品定位不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

1.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-

compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

1.8 运维与发布影响

产品定位会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

1.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文

档必须提醒不要把真实密钥写入脚本、截图或日志。

1.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

1.11 专项展开：品牌治理

用户不会关心内部迁移过程, 只会关心桌面应用是否稳定、能力边界是否清楚、失败提示是否能指导下一步。BAI Work 的产品定位应围绕“本地工程工作台”展开, 以项目为组织单位, 以 BAI Code 为执行核心, 以权限和配置为治理边界。历史来源需要写入白皮书, 但不能成为产品叙事中心。正确做法是说明它完成 BAI Work 独立品牌整理, 已切换对外名称、包名、图标、发布产物和 README 叙事, 同时仍保留若干内部兼容名。这样既不隐瞒迁移现实, 也不让用户误以为当前产品仍依附旧品牌。

从维护视角看, 产品定位需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 产品定位最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 产品定位还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收产品定位的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

1.12 维护者补充说明

维护者在处理产品定位相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把产品定位写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 产品定位还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 产品定位应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

1.13 验收与交付说明

产品定位的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 产品定位章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 产品定位章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

1.14 术语一致性说明

产品定位章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新产品定位相关内容时, 都应同时检查标题、表述、路径、默认值

和限制说明是否仍然一致。

1.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

1.16 小结

从产品定位视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第二章 系统总览与分层架构

2.1 本章摘要

本章说明系统架构在 BAI Work 技术体系中的位置。核心关键词包括 Electron 主壳、Vite/React 渲染层、preload IPC 边界、local /v1/* runtime boundary。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

2.2 背景与目标

本章聚焦系统架构。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

系统架构的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

2.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

2.4 关键设计

在系统架构中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布

形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: Electron 主壳、Vite/React 渲染层、preload IPC 边界、local /v1/* runtime boundary。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

2.5 目标用户与使用场景

桌面客户端开发者、运行时适配开发者、安全评审人员是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

2.6 实现说明

当前实现中, 系统架构不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

2.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

2.8 运维与发布影响

系统架构会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

2.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

2.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

2.11 专项展开：分层协作

系统总览需要把主壳、渲染层、共享契约、运行时适配和资源包放在同一张逻辑图中理解。主进程承担受信任边界, 负责启动本地服务、调用文件系统、管理 runtime、注册 IPC handler 和读取打包资源。渲染进程承担体验边界, 负责工作台、设置页、会话列表、消息流和插件市场。共享层承担契约边界, 让 Provider、settings、policy、endpoint format 和 model profile 有统一类型。资源层承担交付边界, 将 BAI-Code-Runtime、BAI-Code-Official、skills 和品牌资产纳入 builder 控制。

从维护视角看, 系统架构需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 系统架构最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 系统架构还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收系统架构的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包

含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

2.12 维护者补充说明

维护者在处理系统架构相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把系统架构写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 系统架构还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 系统架构应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

2.13 验收与交付说明

系统架构的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 系统架构章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 系统架构章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

2.14 术语一致性说明

系统架构章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新系统架构相关内容时, 都应同时检查标题、表述、路径、默认值

和限制说明是否仍然一致。

2.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

2.16 小结

从系统架构视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第三章 Electron/Vite/React 主壳

3.1 本章摘要

本章说明桌面主壳在 BAI Work 技术体系中的位置。核心关键词包括主进程生命周期、渲染进程体验、preload API、受控 IPC 和 schema 校验。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

3.2 背景与目标

本章聚焦桌面主壳。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

桌面主壳的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事

件。

3.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

3.4 关键设计

在桌面主壳中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible

endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: 主进程生命周期、渲染进程体验、preload API、受控 IPC 和 schema 校验。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

3.5 目标用户与使用场景

前端工程师、Electron 维护者是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

3.6 实现说明

当前实现中, 桌面主壳不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

3.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

3.8 运维与发布影响

桌面主壳会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

3.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

3.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

3.11 专项展开：界面与边界

Electron 主壳的关键不是窗口本身, 而是把本地能力转换为可审计的桌面操作。preload 暴露的 API 应保持窄接口, renderer 不直接访问 Node 文件系统或进程能力。设置保存、runtime 请求、SSE 订阅、Provider 探测、EBAI 安装、技能列表和本地路径打开都通过 IPC handler。handler 再通过 zod schema 或共享类型进行输入约束。这个结构可以降低渲染层被注入脚本滥用本地能力的风险, 也方便后续把危险操作集中到主进程做审计、确认和错误归一化。

从维护视角看, 桌面主壳需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 桌面主壳最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 桌面主壳还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收桌面主壳的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包

含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

3.12 维护者补充说明

维护者在处理桌面主壳相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把桌面主壳写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 桌面主壳还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 桌面主壳应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

3.13 验收与交付说明

桌面主壳的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 桌面主壳章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 桌面主壳章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

3.14 术语一致性说明

桌面主壳章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新桌面主壳相关内容时, 都应同时检查标题、表述、路径、默认值

和限制说明是否仍然一致。

3.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

3.16 小结

从桌面主壳视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第四章 BAI Code Runtime 适配策略

4.1 本章摘要

本章说明 runtime adapter 在 BAI Work 技术体系中的位置。核心关键词包括 baicode CLI 探测、compatibility bridge、线程和 turn 内存存储、CLI 与 OpenAI chat bridge。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

4.2 背景与目标

本章聚焦 runtime adapter。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

runtime adapter 的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模式。

4.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

4.4 关键设计

在 runtime adapter 中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括: baicode CLI 探测、compatibility bridge、线程和 turn 内存存储、CLI 与 OpenAI chat bridge。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

4.5 目标用户与使用场景

Runtime 维护者、集成工程师是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模式。

4.6 实现说明

当前实现中, runtime adapter 不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

4.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后

者属于 BAI Code 生态配置。

4.8 运维与发布影响

runtime adapter 会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

4.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。con-

`tinue_on_error = true` 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

4.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 `BAI_API_KEY=your-key`。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 `trusted workspace scoped`。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

4.11 专项展开: 桥接策略

BAI Work 当前 runtime adapter 的现实目标是“可用且诚实”。它探测 baicode - version, 准备可执行命令, 启动监听 127.0.0.1 的 compatibility server, 再为 renderer 提供 /health、/v1/runtime/info、/v1/skills、/v1/usage、/v1/threads 等路径。线程、turn、item 和事件目前保存在进程内存结构中, 这适合最小可用和开发验证, 但不是长期持久化方案。白皮书必须把这种设计写成过渡桥, 而不是把它宣传为官方 BAI Code service。

从维护视角看, 运行时适配需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 运行时适配最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 运行时适配还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收运行时适配的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

4.12 维护者补充说明

维护者在处理运行时适配相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把运行时适配写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 运行时适配还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 运行时适配应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

4.13 验收与交付说明

运行时适配的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 运行时适配章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 运行时适配章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

4.14 术语一致性说明

运行时适配章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新运行时适配相关内容时, 都应同时检查标题、表述、路径、默认

值和限制说明是否仍然一致。

4.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

4.16 小结

从 runtime adapter 视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 service/session/event/permission/question 实现替换当前 bridge 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第五章 官方 Wheelhouse 与本地 venv

5.1 本章摘要

本章说明官方 wheelhouse 在 BAI Work 技术体系中的位置。核心关键词包括 BAI Code 0.9.1 wheels、Python 3.10-3.13、macOS arm64、Windows x64、离线安装。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

5.2 背景与目标

本章聚焦官方 wheelhouse。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

官方 wheelhouse 的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

5.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

5.4 关键设计

在官方 wheelhouse 中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括: BAI Code 0.9.1 wheels、Python 3.10-3.13、macOS arm64、Windows x64、离线安装。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

5.5 目标用户与使用场景

发布工程师、现场部署工程师是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

5.6 实现说明

当前实现中, 官方 wheelhouse 不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

5.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使

用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

5.8 运维与发布影响

官方 wheelhouse 会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

5.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值

中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

5.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

5.11 专项展开: 依赖策略

Apple Silicon 和 Windows 的 wheelhouse 策略体现的是“资源随包, Python 由目标机提供”。BAI Work 打包官方 BAI Code 0.9.1 wheels 和依赖 wheels, 但不在这些平台打包完整 Python 解释器。启动时如果系统存在 Python 3.10 到 3.13, adapter 会创建数据目录下的 venv, 运行 ensurepip, 再用 pip install --no-index --find-links 从本地 wheelhouse 安装 baicode==0.9.1。这个路径减少联网安装依赖, 但仍要求目标机 Python 可用。

从维护视角看, 官方 wheelhouse 需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 官方 wheelhouse 最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 官方 wheelhouse 还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收官方 wheelhouse 的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

5.12 维护者补充说明

维护者在处理官方 wheelhouse 相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把官方 wheelhouse 写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 官方 wheelhouse 还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 官方 wheelhouse 应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

5.13 验收与交付说明

官方 wheelhouse 的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 官方 wheelhouse 章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 官方 wheelhouse 章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

5.14 术语一致性说明

官方 wheelhouse 章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新官方 wheelhouse 相关内容时, 都应同时检查标题、表述、路径、

默认值和限制说明是否仍然一致。

5.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

5.16 小结

从官方 wheelhouse 视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第六章 Mac Intel x64 Runtime 打包

6.1 本章摘要

本章说明 Mac Intel x64 在 BAI Work 技术体系中的位置。核心关键词包括 BAI-Code-Runtime、CPython 3.11.15 x86_64、bin/baicode、relative path runtime。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

6.2 背景与目标

本章聚焦 Mac Intel x64。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

Mac Intel x64 的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

6.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

6.4 关键设计

在 Mac Intel x64 中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布

形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: BAI-Code-Runtime、CPython 3.11.15 x86_64、bin/baicode、relative path runtime。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

6.5 目标用户与使用场景

macOS 发布维护者、QA 是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

6.6 实现说明

当前实现中, Mac Intel x64 不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

6.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

6.8 运维与发布影响

Mac Intel x64 会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

6.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

6.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

6.11 专项展开：嵌入式运行时

Mac Intel 包采用更封闭的 runtime 策略: resources/bai-code-runtime 内含 BAI Code 0.9.1、CPython 3.11.15 x86_64、site-packages 和 bin/baicode。bin/baicode 的作用是使用相对路径选中随包 Python 和依赖, 避免依赖用户机器的全局 Python。这个策略适合 Intel 目标机, 因为它降低首启失败概率, 也方便发布团队把 runtime 版本与桌面版本绑定。代价是包体更大, 并且 runtime 更新需要走桌面发布流程。

从维护视角看, Mac Intel x64 需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, Mac Intel x64 最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, Mac Intel x64 还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收 Mac Intel x64 的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真

实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

6.12 维护者补充说明

维护者在处理 Mac Intel x64 相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把 Mac Intel x64 写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, Mac Intel x64 还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, Mac Intel x64 应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

6.13 验收与交付说明

Mac Intel x64 的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成

官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, Mac Intel x64 章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, Mac Intel x64 章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

6.14 术语一致性说明

Mac Intel x64 章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新 Mac Intel x64 相关内容时, 都应同时检查标题、表述、路径、默认值和限制说明是否仍然一致。

6.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

6.16 小结

从 Mac Intel x64 视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 service/session/event/permission/question 实现替换当前 bridge 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第七章 跨平台构建与发布

7.1 本章摘要

本章说明构建发布在 BAI Work 技术体系中的位置。核心关键词包括 electron-builder、dmg/zip/nsis/AppImage、artifact naming、update channel、R2 helper scripts。本章内容以仓库当前实现为准,以公开 BAI Code 安装资料为边界,不把未公开的桌面本地 HTTP 协议写成既定官方契约。

7.2 背景与目标

本章聚焦构建发布。BAI Work 的设计起点不是重新发明一个孤立桌面客户端,而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理,对外使用 BAI Work 名称,对运行时使用 BAI Code 叙事,对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换:用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用,维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

构建发布的目标可以概括为三点。第一,保持桌面体验稳定,让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二,尊重 BAI Code 公开文档的边界,公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数,但没有公开桌面本地 HTTP service/session/event/permission/question 协议,因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三,将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束,便于后续团队按相同标准扩展。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置,也允许调试时用环境变量覆盖,但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺,而是过渡层。它在 /health 返回服务状态,在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息,在 /v1/threads 管理内存中的 thread 和 turn,并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事

件。

7.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

7.4 关键设计

在构建发布中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible

endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: electron-builder、dmg/zip/nsis/AppImage、artifact naming、update channel、R2 helper scripts。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

7.5 目标用户与使用场景

Release Manager、DevOps 是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

7.6 实现说明

当前实现中, 构建发布不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

7.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

7.8 运维与发布影响

构建发布会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

7.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

7.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

7.11 专项展开：产物治理

构建发布需要把“能 build”和“能交付”分开。npm run build 只证明 Electron/Vite 产物生成成功, electron-builder 的 mac、win、linux target 才决定安装包结构。macOS 目标包含 dmg 和 zip, Windows 目标包含 NSIS x64, Linux 目标包含 AppImage x64。artifactName 使用 BAI-Work-version-os-arch.ext, 这让用户、更新服务和发布脚本可以通过文件名识别版本、平台和架构。发布通道只允许 stable 或 frontier, 这避免任意字符串污染更新路径。

从维护视角看, 构建发布需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 构建发布最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 构建发布还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收构建发布的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包

含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

7.12 维护者补充说明

维护者在处理构建发布相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把构建发布写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 构建发布还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 构建发布应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

7.13 验收与交付说明

构建发布的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 构建发布章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 构建发布章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

7.14 术语一致性说明

构建发布章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新构建发布相关内容时, 都应同时检查标题、表述、路径、默认值

和限制说明是否仍然一致。

7.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

7.16 小结

从构建发布视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第八章 配置模型与设置迁移

8.1 本章摘要

本章说明配置模型在 BAI Work 技术体系中的位置。核心关键词包括 AppSettingsV1、历史命名兼容、默认端口和数据目录、runtime token、schema 边界。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

8.2 背景与目标

本章聚焦配置模型。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

配置模型的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模式。

8.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

8.4 关键设计

在配置模型中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括: AppSettingsV1、历史命名兼容、默认端口和数据目录、runtime token、schema 边界。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

8.5 目标用户与使用场景

设置页维护者、迁移负责人是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模式。

8.6 实现说明

当前实现中, 配置模型不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

8.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

8.8 运维与发布影响

配置模型会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

8.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

8.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

8.11 专项展开：兼容迁移

配置模型中最容易误解的是历史 RuntimeSettings 这类名称。它们在代码中继续存在, 并不意味着产品仍然以旧品牌对外定位, 而是为了保护旧设置、旧 renderer ABI 和旧测试路径。白皮书应鼓励维护者逐步收敛命名, 但不建议为了视觉洁癖一次性替换所有内部名称。运行时配置包含 binaryPath、port、autoStart、apiKey、baseUrl、providerId、endpointFormat、runtimeToken、dataDir、model、approvalPolicy、sandboxMode 等字段, 每个字段都与用户可见行为有关。

从维护视角看, 配置模型需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 配置模型最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 配置模型还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收配置模型的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包

含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

8.12 维护者补充说明

维护者在处理配置模型相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把配置模型写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 配置模型还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 配置模型应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

8.13 验收与交付说明

配置模型的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 配置模型章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 配置模型章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

8.14 术语一致性说明

配置模型章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新配置模型相关内容时, 都应同时检查标题、表述、路径、默认值

和限制说明是否仍然一致。

8.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

8.16 小结

从配置模型视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第九章 Provider 设置与模型能力

9.1 本章摘要

本章说明 Provider 设置在 BAI Work 技术体系中的位置。核心关键词包括 BAI provider preset、OpenAI-compatible endpoint、endpointFormat、模型能力 profile、API key 凭据治理。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

9.2 背景与目标

本章聚焦 Provider 设置。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

Provider 设置的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

9.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

9.4 关键设计

在 Provider 设置中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括: BAI provider preset、OpenAI-compatible endpoint、endpoint-Format、模型能力 profile、API key 凭据治理。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

9.5 目标用户与使用场景

模型平台工程师、企业集成工程师是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

9.6 实现说明

当前实现中, Provider 设置不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

9.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-

compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

9.8 运维与发布影响

Provider 设置会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

9.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文

档必须提醒不要把真实密钥写入脚本、截图或日志。

9.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

9.11 专项展开：模型治理

Provider 设置的价值在于把模型连接、模型能力和 UI 展示拆开。默认 BAI provider 使用 `https://api.b.ai/v1` 和 `chat_completions` endpoint format, 模型列表由 preset 管理, 每个模型可以声明输入模态、输出模态、工具调用支持、message part 支持、reasoning 能力和上下文窗口。自定义 OpenAI-compatible endpoint 可以复用同一结构, 但必须由用户提供 baseUrl、apiKey 和模型列表。文档不能承诺所有模型都支持工具调用或多模态, 因为能力应由 profile 决定。

从维护视角看, Provider 设置需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, Provider 设置最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, Provider 设置还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收 Provider 设置的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

9.12 维护者补充说明

维护者在处理 Provider 设置相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把 Provider 设置写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, Provider 设置还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, Provider 设置应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

9.13 验收与交付说明

Provider 设置的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, Provider 设置章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, Provider 设置章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

9.14 术语一致性说明

Provider 设置章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新 Provider 设置相关内容时, 都应同时检查标题、表述、路径、默

认值和限制说明是否仍然一致。

9.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

9.16 小结

从 Provider 设置视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十章 本地服务边界与协议现状

10.1 本章摘要

本章说明本地 HTTP 服务在 BAI Work 技术体系中的位置。核心关键词包括 127.0.0.1、/health、/v1/runtime/info、/v1/threads、SSE events、未公开 session/permission/question 契约。本章内容以仓库当前实现为准，以公开 BAI Code 安装资料为边界，不把未公开的桌面本地 HTTP 协议写成既定官方契约。

10.2 背景与目标

本章聚焦本地 HTTP 服务。BAI Work 的设计起点不是重新发明一个孤立桌面客户端，而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理，对外使用 BAI Work 名称，对运行时使用 BAI Code 叙事，对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换：用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用，维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

本地 HTTP 服务的目标可以概括为三点。第一，保持桌面体验稳定，让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二，尊重 BAI Code 公开文档的边界，公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数，但没有公开桌面本地 HTTP service/session/event/permission/question 协议，因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三，将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束，便于后续团队按相同标准扩展。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构，每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error，可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率，但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd，也决定 hooks 是否能以 trusted workspace 形式启用。

10.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

10.4 关键设计

在本地 HTTP 服务中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布

形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: 127.0.0.1、/health、/v1/runtime/info、/v1/threads、SSE events、未公开 session/permission/question 契约。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

10.5 目标用户与使用场景

桌面端开发者、协议设计者是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

10.6 实现说明

当前实现中, 本地 HTTP 服务不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

10.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装

器、桌面快捷方式、开始菜单快捷方式和 `uninstallDisplayName`。

10.8 运维与发布影响

本地 HTTP 服务会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 `Python/site-packages`。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 `venv`, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。`BAI_WORK_BUNDLE_BAI_CODE_RUNTIME` 默认只在 darwin x64 打开, `BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL` 默认在 darwin arm64 和 win32 x64 打开。`artifactName` 采用 `BAI-Work-version-os-arch.ext`, `update channel` 只接受 `stable` 或 `frontier`。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

`hooks TOML` 采用顶层 `enabled` 加多个 `[[hooks]]` 的结构, 每个 hook 包含 `event`、`name`、`command`、`timeout_secs`、`background`、`continue_on_error`, 可选 `condition`。`continue_on_error = true` 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目工作流以 `workspace` 为核心。`thread` 创建时保存 `workspace`、`model`、`mode`、`approvalPolicy`、`sandboxMode`、`createdAt`、`updatedAt`。`workspace` 决定 CLI 子进程 `cwd`, 也决定 `hooks` 是否能以 `trusted workspace` 形式启用。

10.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 `thread/turn/event` 结构, 但不能声称已经获得官方 `session/event/permission/question` 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 `compatibility bridge` 内部实现, 保留 `renderer` 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 `hooks` 审计和 `Provider` 能力探测。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime `venv`、临时会话和 generated artifacts。BAI Work 默认数据目录为 `/.bai-work/data`, BAI Code 用户目录则以 `/.bai` 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 `BAI_API_KEY`、`BAI_BASE_URL`、`BAI_MODEL`

注入子进程环境, 然后执行 `baicode -p`。它适合保持最小可用能力, 但不等同于完整 `session runtime`。CLI `stdout/stderr` 被清理后转成 `assistant text` 或 `tool output`, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

10.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 `BAI_API_KEY=your-key`。
- 涉及 `session`、`event`、`permission`、`question` 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 `hooks` 时必须写明默认 `disabled`, 启用必须 `trusted workspace scoped`。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

10.11 专项展开：协议边界

本地 `/v1/*` 边界是桌面壳自己的稳定边界, 不是官方 BAI Code 已公开的桌面协议。`/v1/runtime/info` 可以告诉 `renderer` 当前 `provider`、`capabilities`、`model`、`sandboxMode` 和 `approvalPolicy`。`/v1/threads` 可以创建 `thread`、追加 `turn`、订阅 `events`。`/v1/sessions` 当前返回 `resume-session` 不可用, `/v1/approvals` 和 `/v1/user-inputs` 在 CLI bridge 下返回 `service unavailable`。这个行为非常关键, 因为它直接告诉 UI 哪些能力不应展示为已完成。

从维护视角看, 本地服务需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 `runtime` 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 本地服务最重要的是可预测性。应用启动后应该能说明 `runtime` 是否可用, `baicode` 是否可探测, `API key` 是否已配置, `Provider base URL` 是否明确, `workspace` 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 `update channel` 都可复现。对安全团队而言, 可预测性意味着 `hooks` 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 本地服务还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。`compatibility bridge` 可以保留, 因为它保护 `renderer` 稳定边界, 但它内部的模拟 `thread store`、CLI 输出整理和 `unavailable route` 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 `hooks` 必须继续采用默认禁用和受

信任 workspace 作用域。

验收本地服务的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

10.12 维护者补充说明

维护者在处理本地服务相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把本地服务写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 本地服务还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 本地服务应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

10.13 验收与交付说明

本地服务的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 本地服务章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 本地服务章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

10.14 术语一致性说明

本地服务章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新本地服务相关内容时, 都应同时检查标题、表述、路径、默认值

和限制说明是否仍然一致。

10.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

10.16 小结

从本地 HTTP 服务视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十一章 安全边界与凭据保护

11.1 本章摘要

本章说明安全边界在 BAI Work 技术体系中的位置。核心关键词包括 API key redaction、本地端口、sandboxMode、approvalPolicy、CSP 与渲染边界。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

11.2 背景与目标

本章聚焦安全边界。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

安全边界的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事

件。

11.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

11.4 关键设计

在安全边界中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible

endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: API key redaction、本地端口、sandboxMode、approvalPolicy、CSP 与渲染边界。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

11.5 目标用户与使用场景

安全工程师、企业管理员是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

11.6 实现说明

当前实现中, 安全边界不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

11.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

11.8 运维与发布影响

安全边界会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

11.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

11.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

11.11 专项展开：凭据与权限

安全边界应从凭据、网络、文件、命令和 hooks 五层看。凭据层要求 API key 只进入受控设置或环境变量, 不进入示例和日志。网络层要求本地服务绑定 127.0.0.1。文件层要求 workspace-write 与 full access 有清晰差异。命令层要求 shell 执行受 sandboxMode 和 approvalPolicy 共同约束。hooks 层要求默认禁用, 且启用只能针对受信任 workspace。任何一个层级被写得含糊, 都会让用户误判应用能访问什么。

从维护视角看, 安全边界需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 安全边界最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 安全边界还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收安全边界的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密

钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

11.12 维护者补充说明

维护者在处理安全边界相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把安全边界写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 安全边界还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 安全边界应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

11.13 验收与交付说明

安全边界的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方

能力。代码层面要求相关实现能在仓库中找到对应文件,例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致,不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时,安全边界章节应帮助其快速判断自己要改哪里。如果要调整用户界面,先看渲染层和共享文案。如果要调整本地 runtime,先看主进程 adapter 和运行时设置。如果要调整模型列表,先看 Provider preset 和能力 profile。如果要调整 EBAI 安装,先看服务实现和测试。如果要调整发布产物,先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索,也能减少跨目录误改。

交付给产品和安全评审时,安全边界章节应说明取舍原因。BAI Work 选择本地 compatibility bridge,是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python,是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用,是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名,是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时,白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制,再考虑扩展路线图。

11.14 术语一致性说明

安全边界章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时,可以写内部兼容名、设置迁移名或旧 ABI 名,但必须说明这些名称不代表当前对外品牌。涉及官方能力时,只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时,必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎,实际上直接影响项目可信度。用户会根据文档判断应用是否成熟,集成方会根据文档判断接口是否稳定,安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称,或把过渡实现写成官方契约,后续沟通成本会明显升高。因此每次更新安全边界相关内容时,都应同时检查标题、表述、路径、默认值和限制说明是否仍然一致。

11.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

11.16 小结

从安全边界视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十二章 EBAI 映射

12.1 本章摘要

本章说明 EBAI 映射在 BAI Work 技术体系中的位置。核心关键词包括 `commands`、`agents`、`rules`、`hooks`、`/.bai` 目标目录、`trusted workspace scoped`。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

12.2 背景与目标

本章聚焦 EBAI 映射。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

EBAI 映射的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 `baicode CLI`、`Python wheel` 安装以及 `OpenAI-compatible LLM` 参数, 但没有公开桌面本地 `HTTP service/session/event/permission/question` 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 `hooks` 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

CLI bridge 的模型调用路径将 `BAI_API_KEY`、`BAI_BASE_URL`、`BAI_MODEL` 注入子进程环境, 然后执行 `baicode -p`。它适合保持最小可用能力, 但不等同于完整 `session runtime`。CLI `stdout/stderr` 被清理后转成 `assistant text` 或 `tool output`, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 `claude-sonnet-4-6`、`gpt-5.2`、`gpt-5-mini`、`gpt-5-nano`、`gemini-3-pro-preview`、`deepseek-v3.2`、`kimi-k2-thinking` 等模型, 并为不同模型设置 `contextWindowTokens` 和输入输出模式。

12.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

12.4 关键设计

在 EBAI 映射中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括: `commands`、`agents`、`rules`、`hooks`、`/.bai` 目标目录、`trusted workspace scoped`。这些重点之间存在明确依赖关系: 打包策略决定可用 `runtime`, `runtime` 能力决定本地 `/v1/*` 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、`hooks`、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

BAI Work 的项目 workflows 以 `workspace` 为核心。`thread` 创建时保存 `workspace`、`model`、`mode`、`approvalPolicy`、`sandboxMode`、`createdAt`、`updatedAt`。`workspace` 决定 CLI 子进程 `cwd`, 也决定 `hooks` 是否能以 `trusted workspace` 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、`runtime venv`、临时会话和 `generated artifacts`。BAI Work 默认数据目录为 `/.bai-work/data`, BAI Code 用户目录则以 `/.bai` 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

12.5 目标用户与使用场景

高级用户、技能生态维护者是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 `runtime` 资源是否随包交付, Python 版本是否满足 `wheelhouse` 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, `hooks` 是否默认禁用, `workspace` 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 `workspace-write`, `shell` 命令需要更高访问级别, `hooks` 则必须以受信任 `workspace` 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

CLI bridge 的模型调用路径将 `BAI_API_KEY`、`BAI_BASE_URL`、`BAI_MODEL` 注入子进程环境, 然后执行 `baicode -p`。它适合保持最小可用能力, 但不等同于完整 `session runtime`。CLI `stdout/stderr` 被清理后转成 `assistant text` 或 `tool output`, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 `claude-sonnet-4-6`、`gpt-5.2`、`gpt-5-mini`、`gpt-5-nano`、`gemini-3-pro-preview`、`deepseek-v3.2`、`kimi-k2-thinking` 等模型, 并为不同模型设置 `contextWindowTokens` 和输入输出模式。

12.6 实现说明

当前实现中, EBAI 映射不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

12.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

12.8 运维与发布影响

EBAI 映射会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

12.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

12.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

12.11 专项展开：生态迁移

EBAI 映射的目标不是原样复制 Claude Code 生态, 而是把可用部分转换为 BAI Code 兼容形态。commands 的 markdown 直接进入 `/.bai/commands`, agents 被包装成 `ebai-agent-* slash command`, rules 被整理成 `ebai-rules skill`, hooks manifest 写入 `/.bai/ebai/hooks/hooks.toml` 并默认禁用。这个过程要保留来源 marker, 因为外部规则和命令需要可追踪。安装器还需要处理重复命令、force 覆盖、dryRun 统计和 includeAll-Rules 范围选择, 让用户知道写入了什么。

从维护视角看, EBAI 映射需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, EBAI 映射最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, EBAI 映射还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收 EBAI 映射的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否

包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

12.12 维护者补充说明

维护者应把 EBAI 视为 BAI Work 对外的能力名称; 源码中保留的 ecc 文件名仅用于兼容既有模块边界, 不应出现在用户界面、安装路径或新文档标题中。

维护者在处理 EBAI 映射相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把 EBAI 映射写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, EBAI 映射还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, EBAI 映射应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

12.13 验收与交付说明

EBAI 映射的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, EBAI 映射章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, EBAI 映射章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

12.14 术语一致性说明

EBAI 映射章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新 EBAI 映射相关内容时, 都应同时检查标题、表述、路径、默认

值和限制说明是否仍然一致。

12.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

12.16 小结

从 EBAI 映射视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十三章 Hooks 安全策略

13.1 本章摘要

本章说明 hooks 安全在 BAI Work 技术体系中的位置。核心关键词包括默认禁用、workspace scoped 启用、shell command 风险、condition mapping、continue_on_error。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

13.2 背景与目标

本章聚焦 hooks 安全。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

hooks 安全的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

13.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

13.4 关键设计

在 hooks 安全中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括：默认禁用、workspace scoped 启用、shell command 风险、condition mapping、continue_on_error。这些重点之间存在明确依赖关系：打包策略决定可用 runtime，runtime 能力决定本地 /v1/* 兼容层能否完整响应，兼容层能力决定渲染层可以展示哪些状态和按钮，安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳，再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

compatibility bridge 不是协议承诺，而是过渡层。它在 /health 返回服务状态，在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息，在 /v1/threads 管理内存中的 thread 和 turn，并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制，使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用，但不提供 BAI Code CLI 的本地工程执行语义。

13.5 目标用户与使用场景

安全评审人员、插件维护者是本章最直接的读者。对这些读者而言，BAI Work 不只是一个聊天窗口，而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付，Python 版本是否满足 wheelhouse 要求，构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理，hooks 是否默认禁用，workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同：文档草拟可以只需要模型调用和文件读取，代码修改需要 workspace-write，shell 命令需要更高访问级别，hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述，目的是让用户在启用能力前知道能力边界，让维护者在实现新功能时知道默认安全姿态。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI，用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code，面向维护者时说明这些兼容接口的边界，防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill，复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要，因为规则包来自外部仓库，如果允许符号链接，可能把用户不期待的本地路径打包到技能目录。

13.6 实现说明

当前实现中, hooks 安全不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

13.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-

compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

13.8 运维与发布影响

hooks 安全会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

13.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文

档必须提醒不要把真实密钥写入脚本、截图或日志。

13.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

13.11 专项展开：执行治理

hooks 安全策略必须比普通技能更严格。技能通常是提示和说明, hooks 会执行命令。BAI Work 生成全局 hooks manifest 时写入 enabled = false, 并生成 README 提醒用户审查。只有当用户在 UI 中选择启用 hooks, 并且当前 workspace 明确存在时, 安装器才会写入 workspace/.bai/hooks.toml 且 enabled = true。这个 workspace scoped 设计避免一个全局安装动作影响所有项目, 也便于团队在单个仓库内做审计和回滚。

从维护视角看, Hooks 安全需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, Hooks 安全最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, Hooks 安全还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收 Hooks 安全的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复

步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

13.12 维护者补充说明

维护者在处理 Hooks 安全相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把 Hooks 安全写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, Hooks 安全还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, Hooks 安全应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

13.13 验收与交付说明

Hooks 安全的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, Hooks 安全章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, Hooks 安全章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

13.14 术语一致性说明

Hooks 安全章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新 Hooks 安全相关内容时, 都应同时检查标题、表述、路径、默认

值和限制说明是否仍然一致。

13.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

13.16 小结

从 hooks 安全视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 service/session/event/permission/question 实现替换当前 bridge 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十四章 工程 workflow 表面

14.1 本章摘要

本章说明工程 workflow 在 BAI Work 技术体系中的位置。核心关键词包括 project-scoped conversations、workspace selection、Git context、side conversations、checkpoints、review surfaces。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

14.2 背景与目标

本章聚焦工程 workflow。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

工程 workflow 的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

14.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

14.4 关键设计

在工程 workflow 中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布

形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: project-scoped conversations、workspace selection、Git context、side conversations、checkpoints、review surfaces。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

14.5 目标用户与使用场景

日常使用者、团队负责人是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

14.6 实现说明

当前实现中, 工程工作流不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

14.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

14.8 运维与发布影响

工程 workflow 会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

14.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

14.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

14.11 专项展开：用户路径

工程 workflow 表面需要支持高频操作而不制造复杂仪式。用户选择 workspace 后发起新会话, composer 根据 runtimeReady 状态决定能否发送, slash menu 提供 new、plan、goal、review、compact、fork、archive、restore、btw 和技能命令。项目列表按 workspace 聚合 thread, 消息流展示 user item、assistant item 和 tool item。这个路径让 BAI Work 更像项目控制台, 而不是一次性问答窗口。

从维护视角看, 工程 workflow 需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 工程 workflow 最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 工程 workflow 还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收工程 workflow 的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实

密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

14.12 维护者补充说明

维护者在处理工程 workflow 相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把工程 workflow 写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 工程 workflow 还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 工程 workflow 应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

14.13 验收与交付说明

工程 workflow 的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官

方能力。代码层面要求相关实现能在仓库中找到对应文件,例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致,不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时,工程 workflow 章节应帮助其快速判断自己要改哪里。如果要调整用户界面,先看渲染层和共享文案。如果要调整本地 runtime,先看主进程 adapter 和运行时设置。如果要调整模型列表,先看 Provider preset 和能力 profile。如果要调整 EBAI 安装,先看服务实现和测试。如果要调整发布产物,先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索,也能减少跨目录误改。

交付给产品和安全评审时,工程 workflow 章节应说明取舍原因。BAI Work 选择本地 compatibility bridge,是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python,是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用,是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名,是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时,白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制,再考虑扩展路线图。

14.14 术语一致性说明

工程 workflow 章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时,可以写内部兼容名、设置迁移名或旧 ABI 名,但必须说明这些名称不代表当前对外品牌。涉及官方能力时,只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时,必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎,实际上直接影响项目可信度。用户会根据文档判断应用是否成熟,集成方会根据文档判断接口是否稳定,安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称,或把过渡实现写成官方契约,后续沟通成本会明显升高。因此每次更新工程 workflow 相关内容时,都应同时检查标题、表述、路径、默认值和限制说明是否仍然一致。

14.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

14.16 小结

从工程工作流视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十五章 写作、自动化与多模态扩展

15.1 本章摘要

本章说明扩展能力在 BAI Work 技术体系中的位置。核心关键词包括 document generation、write workspace、speech/image/music/video capability settings、scheduled tasks、IM channels。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

15.2 背景与目标

本章聚焦扩展能力。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

扩展能力的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事

件。

15.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

15.4 关键设计

在扩展能力中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible

endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: document generation、write workspace、speech/image/music/video capability settings、scheduled tasks、IM channels。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

15.5 目标用户与使用场景

内容工程师、自动化团队是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

15.6 实现说明

当前实现中, 扩展能力不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

15.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

15.8 运维与发布影响

扩展能力会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

15.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

15.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

15.11 专项展开：写作与自动化

写作和自动化能力说明 BAI Work 不只服务代码生成。Write workspace、文档生成、PDF 文本服务、inline completion、inline edit、retrieval RAG、scheduled task、IM channel 和多模态 Provider 设置共同构成更宽的工作台表面。但白皮书应把这些能力描述为项目当前表面和配置方向, 不把所有能力都写成 runtime 已完整支持。尤其是 image、speech、music、video 能力依赖 Provider capability 和设置启用状态。

从维护视角看, 扩展能力需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 扩展能力最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 扩展能力还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收扩展能力的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密

钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

15.12 维护者补充说明

维护者在处理扩展能力相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把扩展能力写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 扩展能力还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 扩展能力应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

15.13 验收与交付说明

扩展能力的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方

能力。代码层面要求相关实现能在仓库中找到对应文件,例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致,不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时,扩展能力章节应帮助其快速判断自己要改哪里。如果要调整用户界面,先看渲染层和共享文案。如果要调整本地 runtime,先看主进程 adapter 和运行时设置。如果要调整模型列表,先看 Provider preset 和能力 profile。如果要调整 EBAI 安装,先看服务实现和测试。如果要调整发布产物,先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索,也能减少跨目录误改。

交付给产品和安全评审时,扩展能力章节应说明取舍原因。BAI Work 选择本地 compatibility bridge,是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python,是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用,是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名,是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时,白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制,再考虑扩展路线图。

15.14 术语一致性说明

扩展能力章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时,可以写内部兼容名、设置迁移名或旧 ABI 名,但必须说明这些名称不代表当前对外品牌。涉及官方能力时,只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时,必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎,实际上直接影响项目可信度。用户会根据文档判断应用是否成熟,集成方会根据文档判断接口是否稳定,安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称,或把过渡实现写成官方契约,后续沟通成本会明显升高。因此每次更新扩展能力相关内容时,都应同时检查标题、表述、路径、默认值和限制说明是否仍然一致。

15.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

15.16 小结

从扩展能力视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十六章 诊断、测试与可维护性

16.1 本章摘要

本章说明验证体系在 BAI Work 技术体系中的位置。核心关键词包括 Vitest、runtime adapter tests、provider tests、packaging checks、secret redaction tests。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

16.2 背景与目标

本章聚焦验证体系。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

验证体系的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模式。

16.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

16.4 关键设计

在验证体系中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户

入口保持一致。

本章涉及的重点包括: Vitest、runtime adapter tests、provider tests、packaging checks、secret redaction tests。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

16.5 目标用户与使用场景

测试工程师、维护者是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模式。

16.6 实现说明

当前实现中, 验证体系不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

16.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

16.8 运维与发布影响

验证体系会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

16.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

16.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

16.11 专项展开：可维护性

验证体系体现项目是否能长期演进。当前仓库包含 runtime adapter tests、provider connection tests、settings normalizer tests、secret redaction tests、packaging config tests、bai-work branding tests、EBAI mapping service tests 和 renderer 组件测试。白皮书应鼓励新增功能先补最小测试, 例如 runtime route 改动应补 adapter 测试, Provider 解析改动应补 profile 和 endpoint 测试, hooks 改动应补 disabled default 和 trusted workspace 测试。

从维护视角看, 验证体系需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 验证体系最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 验证体系还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收验证体系的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包

含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

16.12 维护者补充说明

维护者在处理验证体系相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把验证体系写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 验证体系还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 验证体系应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

16.13 验收与交付说明

验证体系的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 验证体系章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 验证体系章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

16.14 术语一致性说明

验证体系章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似琐碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新验证体系相关内容时, 都应同时检查标题、表述、路径、默认值

和限制说明是否仍然一致。

16.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

16.16 小结

从验证体系视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

第十七章 已知限制与风险登记

17.1 本章摘要

本章说明限制在 BAI Work 技术体系中的位置。核心关键词包括官方桌面契约待确认、Apple/Windows 依赖 Python、CLI bridge 能力有限、hooks 高风险、历史命名。本章内容以仓库当前实现为准, 以公开 BAI Code 安装资料为边界, 不把未公开的桌面本地 HTTP 协议写成既定官方契约。

17.2 背景与目标

本章聚焦限制。BAI Work 的设计起点不是重新发明一个孤立桌面客户端, 而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理, 对外使用 BAI Work 名称, 对运行时使用 BAI Code 叙事, 对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换: 用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用, 维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

限制的目标可以概括为三点。第一, 保持桌面体验稳定, 让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二, 尊重 BAI Code 公开文档的边界, 公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数, 但没有公开桌面本地 HTTP service/session/event/permission/question 协议, 因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三, 将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束, 便于后续团队按相同标准扩展。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

17.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

17.4 关键设计

在限制中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求 Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持

一致。

本章涉及的重点包括：官方桌面契约待确认、Apple/Windows 依赖 Python、CLI bridge 能力有限、hooks 高风险、历史命名。这些重点之间存在明确依赖关系：打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

17.5 目标用户与使用场景

产品负责人、架构评审委员会是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同：文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

17.6 实现说明

当前实现中, 限制不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

17.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

compatibility bridge 不是协议承诺, 而是过渡层。它在 /health 返回服务状态, 在 /v1/runtime/info 返回 provider、capabilities、sandboxMode、approvalPolicy 等信息, 在 /v1/threads 管理内存中的 thread 和 turn, 并通过 text/event-stream 输出 item_updated、tool_call_started、tool_call_finished、assistant_text_delta、turn_completed 等桌面事件。

OpenAI chat bridge 由 BAI_WORK_USE_OPENAI_CHAT_BRIDGE=1 控制, 使用 chat/completions, stream=true, temperature=0, max_tokens=1024。它能验证 OpenAI-

compatible endpoint 路径和模型调用, 但不提供 BAI Code CLI 的本地工程执行语义。

17.8 运维与发布影响

限制会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

历史兼容接口需要文档化治理。当前实现保留若干旧 schema 与 preload ABI, 用于稳定存量设置迁移。白皮书面向外部读者统一使用 BAI Work/BAI Code, 面向维护者时说明这些兼容接口的边界, 防止误删导致迁移失败。

EBAI 安装器会阻止 symlink 内容进入 rules skill, 复制目录时使用 verbatimSymlinks=false 并在发现 symlink 时抛错。这个细节很重要, 因为规则包来自外部仓库, 如果允许符号链接, 可能把用户不期待的本地路径打包到技能目录。

17.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

运行时能力必须明确区分 available、unavailable 和 disabled。available 表示当前路径可用, unavailable 表示能力应该存在但受当前 runtime 或依赖限制不可用, disabled 表示产品在当前模式下不启用该能力。这个区分能帮助 UI 给出准确按钮状态和错误提示。

凭据来源有优先级。运行时会从 BAI credential settings、runtime override、selected provider profile、环境变量 BAI_API_KEY/BAI_BASE_URL/BAI_MODEL 和默认值中解析有效配置。这个模型既允许设置页配置, 也允许调试时用环境变量覆盖, 但生产文档必须提醒不要把真实密钥写入脚本、截图或日志。

17.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

17.11 专项展开：风险登记

已知限制不是削弱产品价值, 而是降低误用风险。当前限制包括官方桌面 HTTP 契约未公开、CLI bridge 不能提供完整 approvals/questions/resume session、Apple Silicon 和 Windows 依赖 Python 3.10-3.13、hooks 启用后会执行本地 shell 命令、内部历史命名仍需治理、内存 thread store 不等同于长期会话存储。把这些限制写清楚, 可以让用户合理预期, 也让路线图有明确验收点。

从维护视角看, 已知限制需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 已知限制最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 已知限制还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收已知限制的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密

钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

17.12 维护者补充说明

维护者在处理已知限制相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把已知限制写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 已知限制还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 已知限制应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

17.13 验收与交付说明

已知限制的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方

能力。代码层面要求相关实现能在仓库中找到对应文件,例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致,不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时,已知限制章节应帮助其快速判断自己要改哪里。如果要调整用户界面,先看渲染层和共享文案。如果要调整本地 runtime,先看主进程 adapter 和运行时设置。如果要调整模型列表,先看 Provider preset 和能力 profile。如果要调整 EBAI 安装,先看服务实现和测试。如果要调整发布产物,先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索,也能减少跨目录误改。

交付给产品和安全评审时,已知限制章节应说明取舍原因。BAI Work 选择本地 compatibility bridge,是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python,是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用,是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名,是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时,白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制,再考虑扩展路线图。

17.14 术语一致性说明

已知限制章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时,可以写内部兼容名、设置迁移名或旧 ABI 名,但必须说明这些名称不代表当前对外品牌。涉及官方能力时,只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时,必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎,实际上直接影响项目可信度。用户会根据文档判断应用是否成熟,集成方会根据文档判断接口是否稳定,安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称,或把过渡实现写成官方契约,后续沟通成本会明显升高。因此每次更新已知限制相关内容时,都应同时检查标题、表述、路径、默认值和限制说明是否仍然一致。

17.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图,不能写成当前能力。这个原则适用于所有章节,也适用于 README 中的编译命令和统计口径。

17.16 小结

从限制视角看, BAI Work 的短期任务是把已经完成的跨平台构建和 BAI Code 接入能力稳定交付给用户,同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后,用正式 service/session/event/permission/question 实现替换当前 bridge 内部逻辑,但保留桌面壳、设置模型和用户工作流的连续性。

第十八章 路线图与演进原则

18.1 本章摘要

本章说明路线图在 BAI Work 技术体系中的位置。核心关键词包括等待官方契约、统一命名、强化安装体验、完善安全审计、扩大平台覆盖。本章内容以仓库当前实现为准，以公开 BAI Code 安装资料为边界，不把未公开的桌面本地 HTTP 协议写成既定官方契约。

18.2 背景与目标

本章聚焦路线图。BAI Work 的设计起点不是重新发明一个孤立桌面客户端，而是在 BAI Code 能力之上提供稳定的本地工作台。项目已经完成 BAI Work 独立品牌整理，对外使用 BAI Work 名称，对运行时使用 BAI Code 叙事，对内部遗留命名采取兼容解释。这个选择的核心价值在于减少用户认知切换：用户看到的是一个面向代码、调试、架构、文档、规划、评审、自动化和工具协同的桌面应用，维护者看到的是一个可以逐步替换 runtime 后端而不破坏渲染层体验的壳层。

路线图的目标可以概括为三点。第一，保持桌面体验稳定，让会话列表、消息流、工具结果、项目选择、模型设置和权限提示在不同运行时状态下有一致入口。第二，尊重 BAI Code 公开文档的边界，公开文档已经覆盖 baicode CLI、Python wheel 安装以及 OpenAI-compatible LLM 参数，但没有公开桌面本地 HTTP service/session/event/permission/question 协议，因此白皮书只能把当前实现写成 BAI Work 自有兼容桥和待确认协议缺口。第三，将构建发布、凭据保护、EBAI 映射和 hooks 安全策略写成可执行工程约束，便于后续团队按相同标准扩展。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构，每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error，可选 condition。continue_on_error = true 降低单个 hook 失败导致工作流中断的概率，但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目工作流以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd，也决定 hooks 是否能以 trusted workspace 形式启用。

18.3 仓库证据

本白皮书基于当前仓库文件和公开 BAI Code 安装资料整理。关键证据包括: README.md 与 README.en.md 对产品定位, runtime 边界和官方契约缺口做了直接描述。electron-builder.config.cjs 定义 extraResources, artifactName, mac/win/linux target 和 BAI_WORK_BUNDLE 的默认逻辑。src/main/runtime/bai-work-adapter.ts 定义 DEFAULT_BAI_CODE_COMMAND, DEFAULT_BAI_BASE_URL, DEFAULT_BAI_MODEL, BAI-Code-Runtime, BAI-Code-Official, wheelhouse venv 准备流程和 /v1/* 兼容服务。src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts 描述 Provider, endpoint format, model profile, multimodal capability 和默认 BAI 预设。src/main/services/ecc-mapping-service.ts 与对应测试描述 EBAI commands, agents, rules, hooks 的安装目标和启用约束。resources/bai-code-runtime/README.md 记录 Intel runtime 使用 BAI Code 0.9.1 与 CPython 3.11.15 x86_64。resources/bai-code-official/README.md 记录 official wheels 来源, Python 3.10-3.13 要求, macOS arm64 和 Windows win_amd64 wheelhouse 覆盖范围。这些文件共同给出一个明确事实: 当前项目已经完成品牌切换、runtime 适配、wheelhouse 打包和 EBAI 映射的主要工程路径, 但仍保留若干历史 ABI 名称。文档在描述这些名称时使用“内部兼容名”或“设置迁移名”, 不把它们当作对外产品名称。

对外文档应避免两类错误。第一类是把未公开的 BAI Code 桌面 HTTP 协议当作既成事实, 这会误导二次开发者直接依赖不存在或未承诺的接口。第二类是把当前兼容桥误读成完整 BAI Code 服务实现, 这会掩盖 CLI bridge 在 approvals、questions、resume session、tool-level semantic events 上的能力差距。白皮书采用更保守的写法: 能从代码和资源中核对的写成当前实现, 只能由路线图推导的写成演进方向, 官方未公开的写成待确认。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

18.4 关键设计

在路线图中, 设计约束来自产品目标、运行时现实和发布形态三方面。产品目标要求桌面壳对用户呈现完整工作台, 包括项目上下文、模型选择、长任务对话、侧边会话、文件引用和生成物管理。运行时现实要求兼容 baicode CLI 与 OpenAI-compatible endpoint, 同时不宣称拥有官方未公开的 session/event/permission/question 协议。发布形态要求

Intel、Apple Silicon、Windows 三类构建拥有不同的 runtime 资源策略, 但用户入口保持一致。

本章涉及的重点包括: 等待官方契约、统一命名、强化安装体验、完善安全审计、扩大平台覆盖。这些重点之间存在明确依赖关系: 打包策略决定可用 runtime, runtime 能力决定本地 /v1/* 兼容层能否完整响应, 兼容层能力决定渲染层可以展示哪些状态和按钮, 安全策略决定哪些命令、hooks、文件访问和凭据路径必须默认收敛。BAI Work 的当前实现倾向于先稳定桌面壳, 再等待 BAI Code 官方契约成熟后替换兼容桥内部实现。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

18.5 目标用户与使用场景

项目管理者、核心维护者是本章最直接的读者。对这些读者而言, BAI Work 不只是一个聊天窗口, 而是本地工程活动的控制台。软件工程师关注是否能在项目目录内启动任务、读取上下文、运行必要命令并整理结果。发布维护者关注 runtime 资源是否随包交付, Python 版本是否满足 wheelhouse 要求, 构建产物命名是否稳定。安全人员关注 API key 是否只作为凭据处理, hooks 是否默认禁用, workspace 作用域是否明确。

典型使用场景包括本地代码修改、仓库评审、技术方案草拟、文档生成、调试辅助、自动化任务和跨渠道任务触发。不同场景对权限的要求不同: 文档草拟可以只需要模型调用和文件读取, 代码修改需要 workspace-write, shell 命令需要更高访问级别, hooks 则必须以受信任 workspace 为前提。白皮书把这些场景拆开描述, 目的是让用户在启用能力前知道能力边界, 让维护者在实现新功能时知道默认安全姿态。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

18.6 实现说明

当前实现中, 路线图不是孤立模块, 而是跨主进程、渲染进程、共享设置和资源目录共同组成。主进程负责 runtime 探测、compatibility server、IPC handler、发布资源定位和本地文件写入。渲染进程负责工作台 UI、设置页、slash command 菜单、插件技能市场和用户交互。共享层负责类型、schema、Provider profile、endpoint format、默认值和路径规范。资源目录提供 bundled runtime、official wheelhouse、skills 和品牌资产。

这种分层有一个实用优点: 当 BAI Code 官方服务契约尚未公开时, 渲染层不必频繁改动。它仍然调用稳定的本地 /v1/* 桌面边界, 主进程可以在内部从 CLI bridge、OpenAI chat bridge 或将来的官方 service bridge 之间切换。这个边界也有代价: 兼容层必须诚实暴露 unavailable、disabled 或 unsupported 状态, 避免 UI 把尚未实现的 approvals、questions、resume-session 能力伪装成可用。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

18.7 安全与合规考量

安全策略采用最小惊讶原则。API key 不写入文档示例, 不写入日志, 不提交仓库, 示例统一使用 BAI_API_KEY=your-key。本地服务默认绑定 127.0.0.1, 避免暴露到局域网。运行时信息只报告 apiKeyConfigured 这样的布尔值, 不回显密钥。CLI 输出清理阶段会对当前配置中的 API key 做替换, 以降低错误展示凭据的风险。

EBAI hooks 是最高风险能力之一, 因为 hooks 本质上会执行本地 shell 命令。BAI Work 的安装器默认只在用户目录生成 disabled manifest, 并写入提示文档。启用 hooks 时必须提供 workspace, 结果写入该 workspace 下的 .bai/hooks.toml, 且 enabled = true 只存在于该受信任 workspace。这个策略不能消除 hooks 自身风险, 但能降低“安装即全局执行”的误用概率。

Provider profile 把模型列表和能力描述从 UI 字符串中抽出。默认 BAI preset 包含 claude-sonnet-4-6、gpt-5.2、gpt-5-mini、gpt-5-nano、gemini-3-pro-preview、deepseek-v3.2、kimi-k2-thinking 等模型, 并为不同模型设置 contextWindowTokens 和输入输出模态。

release 脚本和 builder 配置共同决定最终交付形态。macOS 包含 hardenedRuntime、entitlements、notarization hook、DMG/ZIP 目标和图标资产, Windows 包含 NSIS 安装器、桌面快捷方式、开始菜单快捷方式和 uninstallDisplayName。

18.8 运维与发布影响

路线图会直接影响构建和现场部署。Mac Intel x64 可以依赖随包交付的 BAI-Code-Runtime, 该资源内含 baicode 入口和相对路径 Python/site-packages。Mac Apple Silicon arm64 与 Windows x64 走 official wheelhouse 路线, 目标机如果没有 Python 3.10-3.13, 应用无法创建本地 venv, 需要用户安装 Python 或在设置中指定已有 baicode 可执行文件。

发布侧需要关注 electron-builder 的条件打包逻辑。BAI_WORK_BUNDLE_BAI_CODE_RUNTIME 默认只在 darwin x64 打开, BAI_WORK_BUNDLE_BAI_CODE_OFFICIAL 默认在 darwin arm64 和 win32 x64 打开。artifactName 采用 BAI-Work-version-os-arch.ext, update channel 只接受 stable 或 frontier。白皮书不把这些变量写成用户必须配置的运行条件, 而是写成发布工程可调整的构建参数。

hooks TOML 采用顶层 enabled 加多个 [[hooks]] 的结构, 每个 hook 包含 event、name、command、timeout_secs、background、continue_on_error, 可选 condition。continue_on_error = true 降低单个 hook 失败导致 workflow 中断的概率, 但也要求用户通过日志或诊断界面理解 hook 是否真正执行成功。

BAI Work 的项目 workflow 以 workspace 为核心。thread 创建时保存 workspace、model、mode、approvalPolicy、sandboxMode、createdAt、updatedAt。workspace 决定 CLI 子进程 cwd, 也决定 hooks 是否能以 trusted workspace 形式启用。

18.9 边界、限制与后续演进

当前最大限制仍是官方桌面服务契约未公开。BAI Work 可以调用 baicode CLI, 可以用 OpenAI-compatible chat completions 做有限 bridge, 可以为渲染层模拟 thread/turn/event 结构, 但不能声称已经获得官方 session/event/permission/question 协议。这个限制影响恢复会话、细粒度权限审批、问题回传、真实工具事件映射、长期存储和多客户端同步。

后续演进应遵循两条线。第一条是产品线: 统一品牌语言、简化设置项、提高错误可读性、完善缺少 Python 或缺少 API key 时的引导。第二条是工程线: 在官方契约发布后替换 compatibility bridge 内部实现, 保留 renderer 边界, 增加协议测试, 增加跨平台安装 smoke test, 补齐 hooks 审计和 Provider 能力探测。

数据目录与安装目录必须分离。安装目录随应用升级替换, 数据目录承载用户设置、runtime venv、临时会话和 generated artifacts。BAI Work 默认数据目录为 /.bai-work/data, BAI Code 用户目录则以 /.bai 为目标, 二者服务不同目的。前者属于桌面应用状态, 后者属于 BAI Code 生态配置。

CLI bridge 的模型调用路径将 BAI_API_KEY、BAI_BASE_URL、BAI_MODEL 注入子进程环境, 然后执行 baicode -p。它适合保持最小可用能力, 但不等同于完整 session runtime。CLI stdout/stderr 被清理后转成 assistant text 或 tool output, 这意味着复杂工具语义只能以摘要形式出现在桌面消息流。

18.10 工程判定清单

- 对外文案必须使用 BAI Work 和 BAI Code, 历史兼容名称只作为兼容名解释。
- 所有示例凭据必须使用占位符, 例如 BAI_API_KEY=your-key。
- 涉及 session、event、permission、question 的官方协议时必须写明“待官方公开契约确认”。
- 涉及 hooks 时必须写明默认 disabled, 启用必须 trusted workspace scoped。
- 涉及 Apple Silicon 或 Windows runtime 时必须写明 Python 3.10-3.13 要求。

18.11 专项展开：演进原则

路线图应避免空泛愿景, 采用可验证里程碑。第一阶段是文档和错误提示收敛, 让用户知道如何安装 Python、配置 BAI_API_KEY、选择 provider 和定位 runtime 状态。第二阶段是 runtime bridge 强化, 包括持久化会话、协议测试和更准确的事件映射。第三阶段是等待官方 BAI Code 桌面 service 契约, 用正式 session/event/permission/question 实现替换当前过渡逻辑。第四阶段是安全审计和企业交付, 包括 hooks 审计、凭据扫描、发布签名和更新通道治理。

从维护视角看, 路线图需要持续回答四个问题。第一, 用户看到的能力是否已经由当前代码或资源支撑。第二, 失败时是否给出可执行的恢复路径。第三, 默认配置是否保护本地文件、凭据和工作区。第四, 新增能力是否会扩大打包、启动、权限或协议依赖。BAI Work 当前处在品牌切换和 runtime 适配并行推进阶段, 因此文档必须把“当前实现”“兼容层”“待确认官方契约”和“路线图”分开写。只有这样, 后续维护者才能在不破坏用户体验的前提下替换内部实现。

对用户而言, 路线图最重要的是可预测性。应用启动后应该能说明 runtime 是否可用, baicode 是否可探测, API key 是否已配置, Provider base URL 是否明确, workspace 是否已选择, 以及当前权限是否允许文件修改和命令执行。对发布团队而言, 可预测性意味着构建参数、资源目录、产物名称、签名状态和 update channel 都可复现。对安全团队而言, 可预测性意味着 hooks 不会在用户不知情的情况下执行, 凭据不会出现在日志或文档中, 本地服务不会监听外部网卡。

在实现层面, 路线图还应避免把历史兼容当作永久架构。兼容名可以保留, 因为它们保护旧设置和旧 ABI, 但新增文档、错误提示和用户界面应尽量使用 BAI Work/BAI Code 术语。compatibility bridge 可以保留, 因为它保护 renderer 稳定边界, 但它内部的模拟 thread store、CLI 输出整理和 unavailable route 必须随着官方契约成熟而替换。EBAI 映射可以保留, 因为它帮助用户迁移已有规则资产, 但 hooks 必须继续采用默认禁用和受信任 workspace 作用域。

验收路线图的方式也应具体。文档验收看是否列明路径、默认值、限制和恢复步骤。代码验收看是否有单元测试覆盖成功路径和失败路径。发布验收看目标平台是否包含预期资源, Intel 包是否包含 BAI-Code-Runtime, Apple Silicon 和 Windows 包是否包含 BAI-

Code-Official, wheelhouse 是否覆盖 cp310 到 cp313。安全验收看是否没有真实密钥, 是否没有把 hooks 写成默认启用, 是否没有把未公开协议写成官方承诺。

这些要求共同形成 BAI Work 的文档纪律: 宁可把不确定性写清楚, 也不为了完整感补齐不存在的协议细节。宁可把跨平台差异写复杂一点, 也不让 Apple Silicon 或 Windows 用户误以为无需 Python。宁可把 hooks 风险重复说明, 也不让用户把外部命令包当作普通提示词资产。宁可承认内部命名仍在迁移, 也不为了品牌统一在代码层做高风险重命名。

18.12 维护者补充说明

维护者在处理路线图相关变更时, 应先确认变更是否直接服务用户请求或当前路线图。BAI Work 已经有较多跨层能力, 包括桌面 UI、运行时桥接、Provider 设置、技能市场、EBAI 安装、发布脚本和多平台资源。任何看似简单的改动, 都可能影响打包产物、首启流程、设置迁移或安全默认值。比如修改默认模型会影响 Provider preset、runtime config、composer 默认值和文档示例。修改数据目录会影响 venv 缓存、工作区默认路径和用户已有状态。修改 hooks 事件映射会影响外部命令执行时机。

因此, 白皮书把路线图写成可维护对象, 而不是一次性说明文字。每个章节都应帮助读者回答“当前版本如何工作”“为什么这样设计”“哪里不能承诺”“如何验证”“下一步如何演进”。这种写法会比营销式介绍更长, 但能减少后续误解。特别是在 BAI Code 官方桌面 service 契约尚未公开的阶段, 文档越需要克制: 可以写 BAI Work 当前有 compatibility bridge, 可以写 renderer 依赖 /v1/* 桌面边界, 可以写某些 route 返回 unsupported 或 unavailable, 但不能写官方已经承诺这些 HTTP path。

在团队协作中, 路线图还需要形成检查清单。提交前检查是否修改了写入范围之外的文件, 是否引入真实凭据, 是否把示例 key 写成真实格式, 是否使用了不适合 XeLaTeX 的字符, 是否让 README 的编译命令与实际目录不一致, 是否让 references.bib 引用不存在来源。发布前检查包内是否存在预期资源, Intel 包是否有 BAI-Code-Runtime, arm64 和 Windows 包是否有 BAI-Code-Official, Windows 安装器是否保留用户 app data, macOS 包是否使用正确图标和 entitlements。

从用户教育角度看, 路线图应使用直接语言。用户不需要知道每一个内部函数名, 但需要知道配置 API key 后会发生什么, 缺少 Python 时该做什么, 开启 hooks 前要信任什么, 自定义 Provider 需要满足什么接口形态, 为什么某些权限或问题功能当前不可用。白皮书把这些解释写入正文, 是为了让产品、开发、测试、安全和发布团队在同一份材料上形成共识。

18.13 验收与交付说明

路线图的交付验收应同时覆盖文档、代码、打包和安全四个层面。文档层面要求章节描述与仓库事实一致, 不出现聊天痕迹, 不出现真实凭据, 不把待确认协议写成官方能力。代码层面要求相关实现能在仓库中找到对应文件, 例如运行时适配应能追溯到主进程 adapter, Provider 能力应能追溯到共享设置和 preset, EBAI 行为应能追溯到安装服务和测试。打包层面要求目标平台资源与构建配置一致, 不把 Intel runtime 和 official wheelhouse 的适用平台混写。安全层面要求 hooks、shell、workspace、API key 和本地端口都有明确默认值和风险说明。

交付给新开发者时, 路线图章节应帮助其快速判断自己要改哪里。如果要调整用户界面, 先看渲染层和共享文案。如果要调整本地 runtime, 先看主进程 adapter 和运行时设置。如果要调整模型列表, 先看 Provider preset 和能力 profile。如果要调整 EBAI 安装, 先看服务实现和测试。如果要调整发布产物, 先看 electron-builder 配置和 release 脚本。这样的路径说明能减少盲目搜索, 也能减少跨目录误改。

交付给产品和安全评审时, 路线图章节应说明取舍原因。BAI Work 选择本地 compatibility bridge, 是为了在官方桌面服务契约尚未公开时保持桌面体验连续。BAI Work 选择 wheelhouse 加目标机 Python, 是为了在 Apple Silicon 和 Windows 上复用官方 BAI Code wheel 安装模型。BAI Work 选择 hooks 默认禁用, 是因为外部 hooks 会执行本地命令。BAI Work 保留部分历史内部命名, 是因为设置迁移和 ABI 稳定比一次性重命名更重要。

交付后的维护还需要周期性复查。每当 BAI Code 官方文档更新、wheel 版本变化、Provider 模型列表变化、Electron 或 electron-builder 升级、Windows 安装器策略变化、macOS 签名要求变化、EBAI 上游结构变化时, 白皮书都应同步更新。否则文档会从工程证据变成历史快照。维护者应优先更新事实性段落、命令、路径、默认值和已知限制, 再考虑扩展路线图。

18.14 术语一致性说明

路线图章节使用的术语应与整份白皮书保持一致。对外统一写 BAI Work、BAI Code、BAI provider、BAI-Code-Runtime、BAI-Code-Official。涉及历史实现时, 可以写内部兼容名、设置迁移名或旧 ABI 名, 但必须说明这些名称不代表当前对外品牌。涉及官方能力时, 只写公开文档已经覆盖的 baicode CLI、Python wheel 安装和 OpenAI-compatible LLM 参数。涉及桌面本地 service、session、event、permission、question 时, 必须写成 BAI Work 当前兼容桥或待官方公开契约确认。

这种术语纪律看似细碎, 实际上直接影响项目可信度。用户会根据文档判断应用是否成熟, 集成方会根据文档判断接口是否稳定, 安全评审会根据文档判断风险是否被承认。如果同一能力在不同章节被写成不同名称, 或把过渡实现写成官方契约, 后续沟通成本会明显升高。因此每次更新路线图相关内容时, 都应同时检查标题、表述、路径、默认值和

限制说明是否仍然一致。

18.15 更新原则

白皮书后续更新应坚持证据优先。新增事实必须能追溯到代码、配置、资源目录、测试或公开文档。无法验证的判断只能写成假设、风险或路线图, 不能写成当前能力。这个原则适用于所有章节, 也适用于 README 中的编译命令和统计口径。

18.16 小结

从路线图视角看, BAI Work 的短期任务是已完成的跨平台构建和 BAI Code 接入能力稳定交付给用户, 同时把不确定协议边界暴露给维护者。长期任务是在 BAI Code 官方桌面服务契约明确后, 用正式 `service/session/event/permission/question` 实现替换当前 `bridge` 内部逻辑, 但保留桌面壳、设置模型和用户工作流的连续性。

参考资料

- BAI Work 仓库 README.md 和 README.en.md。
- BAI Work 仓库 electron-builder.config.cjs。
- BAI Work 仓库 src/main/runtime/bai-work-adapter.ts。
- BAI Work 仓库 src/shared/app-settings-types.ts, src/shared/app-settings-provider.ts, src/shared/model-provider-presets.ts。
- BAI Work 仓库 src/main/services/ecc-mapping-service.ts。
- BAI Work 仓库 resources/bai-code-runtime/README.md 和 resources/bai-code-official/README.md。
- 公开 BAI Code 文档与官方安装脚本, 详见 references.bib。